

HPC101 期末大作业答辩

鲲鹏 920F 三算子优化：CONV · TRSM · ZGEMM

浙江大学

齐思航 王传宇

2026-09-06

总览

- 平台：鲲鹏 920F 开发集群（ARMv9 SVE2，608 核 / 32 NUMA $\times$ 38 核 @ 2.0 GHz）
- 技术栈：C + OpenMP，NEON SIMD / 循环分块 / BLAS 库编排 / 环境自适应线程
- 分工：CONV、ZGEMM —— 齐思航；TRSM —— 王传宇

题目	负责人	基线 GFLOPS	最终 GFLOPS	加速比
CONV	齐思航	12.95	748–886	58–68 $\times$
TRSM	王传宇	1.7–2.7	102–213	48–115 $\times$
ZGEMM	齐思航	5.7–49	160–207	2.5–24 $\times$

三题正确性全部通过官方校验；最终区间取自各题性能图。

题目一 · CONV

二维卷积优化 · 齐思航

问题与基线

- 任务：给图像与卷积核，求二维卷积输出（4 组不同尺寸的用例）
- 基线：朴素四重循环 + 外层 OpenMP 并行

配置	GFLOPS
OMP 1 线程	1.00
OMP 38 线程	12.95

- 环境约束：donau 调度下单 NUMA 绑定会使 38 线程坍缩到单核，故按自由环境 38 线程评测

V1 优化（一）：编译器友好化

- 局部累加：float sum 临时量，消除对输出数组的反复读写
- restrict：声明 input / kernel / output 不重叠，解放寄存器分配与向量化
- OpenMP 静态调度：按输出行分配线程，降低调度开销
- 指针提升：预计算行指针 inputRow / kernelRow，内层免乘法寻址

V1 优化 (二): NEON SIMD 四列同算

- float32x4_t 思路: 一次计算连续 4 个输出列

```
float sum0, sum1, sum2, sum3;
for (int jk = 0; jk < kernelHeight; ++jk)
    for (int ik = 0; ik < kernelWidth; ++ik) {
        float w = kernel[jk * kernelWidth + ik];
        sum0 += input[(j+jk)*W + (i+ik)] * w;
        sum1 += input[(j+jk)*W + (i+ik+1)] * w;
        sum2 += input[(j+jk)*W + (i+ik+2)] * w;
        sum3 += input[(j+jk)*W + (i+ik+3)] * w;
    }
```

- V1 后: 12.95 → **165.57 GFLOPS** (12.8×)

V2 优化（一）：输出列分块 + 四行融合

- 输出列分块：一次处理 16 列 = 4 个 NEON 向量 $\times$ 4 组累加器
- 规模探索：8 列并行不足，20/32/64 列寄存器压力过大 —— 16 列是平衡点
- 四行融合：相邻输出行的卷积核窗口部分重叠，一次加载复用于 4 行

```
CONV_ROW4_VP_FMA(s0, v, w, 0);  
CONV_ROW4_VP_FMA(s1, v, w, 1);  
CONV_ROW4_VP_FMA(s2, v, w, 2);  
CONV_ROW4_VP_FMA(s3, v, w, 3);
```

- 实测四行优于二行/六行，单步 +22% $\rightarrow$ **621.42 GFLOPS**

V2 优化（二）：输入滑动窗口

- 相邻 ik 的输入窗口高度重叠，重复加载浪费带宽：

```
ik = 0: input[0..3]
```

```
ik = 1: input[1..4]
```

```
ik = 2: input[2..5]
```

- 三步滑窗：加载 5 个相邻向量 $\rightarrow$ vextq_f32 生成 4 个滑动窗口 $\rightarrow$ 分别完成 4 个卷积位置
- 第二轮优化后：**809.31 GFLOPS** ($62\times$ vs 基线；四用例 748–886，全部 PASS)

CONV 最终性能

===== CONV测试 =====						
ImageSize	kernelSize		Time(ms)	GFLOPS	最大误差	校验
4096 x 6144	39 x	39	100.70	748.4792	0.00e+00	PASS
[qisihang@login01 conv]\$./conv2d_test 6144 4096 41 41 3						
===== CONV测试 =====						
ImageSize	kernelSize		Time(ms)	GFLOPS	最大误差	校验
6144 x 4096	41 x	41	93.95	885.9192	0.00e+00	PASS
[qisihang@login01 conv]\$./conv2d_test 4256 6390 55 55 3						
===== CONV测试 =====						
ImageSize	kernelSize		Time(ms)	GFLOPS	最大误差	校验
4256 x 6390	55 x	55	211.54	761.4527	0.00e+00	PASS
[qisihang@login01 conv]\$./conv2d_test 6390 4256 81 81 3						
===== CONV测试 =====						
ImageSize	kernelSize		Time(ms)	GFLOPS	最大误差	校验
6390 x 4256	81 x	81	404.82	854.1302	0.00e+00	PASS

四用例 748–886 GFLOPS, 全部 PASS

题目二 · TRSM

三角方程组求解优化 · 王传宇

问题与基线

- 任务：行主序下求解 $LX = B$ (L 为 $M \times M$ 下三角), 计算量约 M^2N
- 三用例覆盖三种形状: 512×19968 (宽)、 2432×17024 (方)、 17024×512 (高)
- 基线：逐列前代消元 + 列间 OpenMP $\rightarrow 2.12 / 2.74 / 1.69$ GFLOPS
- 瓶颈：行主序下按列跨步访存 B 、标量点积无法向量化、列内依赖链限制并行

优化一：分块算法 + BLAS 库编排

- L 沿对角切成宽度 512 的条带，串行推进，每步两段：
 - **对角段**：按列切片并行，每线程一次单线程 `cblas_dtrsm` 解出自己的列条带
 - **更新段**：Schur 补更新 $B_{\text{rest}} - = L_{\text{rest},k} \times X_k$ ，全部转化为规则 `dgemm`
- 形状自适应：目标块扁则按列分割广播 L ，高则按行分割广播 X_k
- 库调用单线程化 (`BlasSetNumThreads(1)`)，外层 OpenMP 编排 8 个并发库调用
- 健康环境即达 96–213 GFLOPS

更新段细节：三路连续访存的规则 GEMM

- 更新段占高瘦用例 **约 97%** 计算量 ($1 - \frac{512}{17024}$), 是性能主战场

```
for (r = 0; r < below; ++r)           // 目标 B 的第 r 行
    for (j = 0; j < kb; ++j) {
        double w = L_ik[r][j];         // L: 按行连续
        for (c = 0; c < n; ++c)
            B[r][c] -= w * X[j][c];    // B、X: 按行连续
    }
```

示意：与库 `cblas_dgemm` 内核等价的循环序

- 三操作数全部按行流动：缓存行满载，无跨步浪费
- 内层“标量 × 向量”是 SIMD 完美形状 (SVE 512-bit = 8 double/指令)
- 基线对照：累加目标只有一个标量 → 依赖链锁死、跨步访存
- L 面板全程只流一遍；基线每解一列全量重读 L

优化二之发现：合流坍缩

- 现象：官方命令（38 线程 + numactl -N 1 单 NUMA）运行数秒后，全部线程被强制调度到同一物理核
 - 吞吐下降 20–30 倍，且粘性：同进程后续单线程也被降速
- 机理：集群亲和域内核对“掩码内严重过订阅”的强制合流，评估周期约 2 s， ≤ 8 线程不触发
- 两个硬约束：
 - 短探针会被延迟生效期骗过（探针时正常，正式工作开始后坍缩）
 - “先试并行、失败退串行”不可行——试探本身触发粘性污染

v10 自适应方案

- **正式工作封顶 8 线程**：实测不触发合流，可借掩码内空闲核获得真实 $8\times$ 加速；即使被合流也只是退化为串行水平（最坏 8.3 s，无防护版本 95 s+）
- **一次性决策，绝不回退**：与正式工作同构的 8 线程探针（“测得即跑得”）
- **双信号判定**：
 - `sched_getcpu()` 采样：8 线程每 20 ms 上报所在核，连续 5 个采样点全部同核 → 串行
 - FMA 吞吐比：低于 3 选串行，不低于 6 放行并行，模糊带二次采样取最小

TRSM 最终性能：同条件对比

用例 (M×N)	基线	v10 第 1 次	v10 第 2 次	加速比
512×19968	2.12 GFLOPS	81.00	102.82	48.4×
2432×17024	2.74 GFLOPS	213.12	190.85	77.7×
17024×512	1.69 GFLOPS	184.24	194.45	114.8×

- 总时间 126.78 s → 1.34 s (**94.5×**)
- 官方形态自适应：中等天气前两案并行、第三案探针转串行；毒化天气全程串行（有界，无灾难）
- 串行兜底分支 23.1 / 29.6 / 35.0 GFLOPS，仍为基线 11–20 倍

TRSM 最终版运行输出 (健康环境)

===== TRSM测试 =====					
M	N	Time (ms)	GFLOPS	最大误差	校验

512 x 19968		50.91	102.8177	3.33e-16	PASS
===== TRSM测试 =====					
M	N	Time (ms)	GFLOPS	最大误差	校验

2432 x 17024		527.58	190.8537	4.44e-16	PASS
===== TRSM测试 =====					
M	N	Time (ms)	GFLOPS	最大误差	校验

17024 x 512		763.11	194.4502	7.77e-16	PASS

三用例 102.8 / 190.9 / 194.5 GFLOPS, 全部 PASS

题目三 · ZGEMM

复数矩阵乘法优化 · 齐思航

问题与基线

- 任务: $C = \alpha AB + \beta C$, A、B、C 为复数矩阵, 行主序、不转置 (竞赛路径)
- 基线: 朴素三重循环 $\rightarrow$ 49.03 / 9.81 / 5.73 GFLOPS
- 复数交错存储 (实虚交替) 是后续 SIMD 优化的主要障碍

V1-V2: 竞赛路径分发 + 循环重排

- V1: 行主序、不转置的竞赛路径, 分发到专优化实现

```
if (Order == CblasRowMajor &&  
    TransA == CblasNoTrans &&  
    TransB == CblasNoTrans) {  
    zgemm_row_notrans_real_simd(...);  
    return;  
}
```

- V2: i-j-k 改为 i-k-j, B 数组按行连续访问, 缓存命中提升

```
for (i) for (k) {  
    const complex scaled_a = alpha * A[i][k];  
    for (j = 0; j < N; ++j)  
        C[i][j] += scaled_a * B[k][j];  
}
```

V3-V4: 分块 + 实虚分离

- V3: C 按 8×128 分块计算、一次性写回; 局部累加器驻留

```
double acc_re[8][128];  
double acc_im[8][128];
```

- V4: 复数展开为实/虚两部, omp simd 生成 128 位 SIMD

```
acc_re += ar * br - ai * bi;  
acc_im += ar * bi + ai * br;  
#pragma omp simd  
for (j = 0; j < jmax; ++j) ...
```

- 实测: 49.03 / 9.81 / 5.73 $\rightarrow$ **110.87 / 106.14 / 98.56 GFLOPS**

V5–V6: 实虚部打包初试 + perf 定位

- V5: A/B 拆为连续实虚数组, 消除交错加载 → 初测 **122.88 / 137.72 / 135.89 GFLOPS**
- V6: perf stat 热点分析, 确认方向后再深化
 - IPC 0.53、后端停顿 79%、cache miss 约 5.5e10
 - 结论: **memory-bound**, 瓶颈在后端访存等待, 而非 SIMD 未生效
 - 方法论: 不盲目加微内核宽度, 转向访存优化

V7-V8: 打包深化与终验

- V7 B 打包 (单 NUMA 38 线程, 对照实测): Case1 1.04×、Case2 1.16×、Case3 1.26×
 - 后端停顿仍高: 降低了拆分开销, 未消除内存层级瓶颈 → 保留为默认路径
- V8 A 打包: 计算核心只访问连续实数数组
- 最终配置: 8×128 分块 + A/B 实虚部打包 + omp simd
- 终验 (单 NUMA 38 线程): **122.88 / 137.72 / 135.90 GFLOPS**, 误差 7.90e-13 / 7.63e-13 / 1.48e-12, 全部 PASS

V11 / V13：探索与放弃

- V11 M 方向分块 (C 分块默认 8×128 , 提高 A 行块复用):

配置	Case1	Case2	Case3
M=4	102.04	110.49	102.02
M=16	130.18	136.72	142.43

- M=4 明显降低复用效率, 淘汰; M=16 在 Case2 未稳定超过默认 $\rightarrow$ 保守保 M=8
- V13 K 方向分块 (64 段): 120.96 / 137.81 / 127.87, backend idle 74.68% $\rightarrow$ 74.31%
无稳定提升——循环控制开销抵消缓存收益, 弃用

V15 诊断 + ZGEMM 最终性能

- -fopt-info-vec 确认内层已 128 位向量化, 限制在 backend memory stall
- 登录节点复测 (同代码, 误差指纹一致): **160.24 / 206.47 / 207.19 GFLOPS**

```
[qisihang@login04 zgemm]$ ./zgemm_test 7427 7427 256 3
7427 x 7427 x 256      704.98      160.2434      7.90e-13      PASS
[qisihang@login04 zgemm]$ ./zgemm_test 14848 14848 256 3
14848 x 14848 x 256    2186.83     206.4675      7.63e-13      PASS
[qisihang@login04 zgemm]$ ./zgemm_test 37360 8192 512 3
37360 x 8192 x 512     6050.46     207.1899      1.48e-12      PASS
```

总结与方法论

- 三题全部通过正确性校验，加速比 2.5–115×
- 测量纪律：min-of-N、同环境对比、perf 先定位再动手
- 优化针对实测瓶颈，不追求“看起来更高级”的写法（ZGEMM 向量化尾段两次失败即回退）
- 环境适应：TRSM 的同构探针一次决策，把最坏情形结构性约束在串行水平
- 生成式 AI 工具声明：DeepSeek Harness（glm-5.3）辅助，全部方案经实验验证

谢谢聆听

欢迎提问